

Full Study Notes

Introduction to Node.js Event Loop and Asynchronous Programming

=====

What is the Event Loop?

The Event Loop is the core of Node.js, allowing it to handle multiple operations without blocking, even though it runs on a single thread. Think of it like a factory worker that keeps checking different sections in a fixed order.

Phases of the Event Loop

The Node.js event loop runs in six phases, each with its own type of tasks. The phases cycle like this:

- * Timers: Callbacks scheduled by `setTimeout()` and `setInterval()`.
- * Pending Callbacks (I/O callbacks): System-level callbacks, such as errors from TCP or file system.
- * Idle/Prepare (internal): Used internally by Node.js, not interacted with directly.
- * Poll (MOST IMPORTANT): Executes I/O callbacks, waits for new I/O events, and can block if nothing else to do.
- * Check: `setImmediate()` callbacks.
- * Close Callbacks: Socket close events and cleanup events.

Practical Examples

- * Timers Phase:

```
```javascript
```

```
setTimeout(() => {
 console.log("Timer executed");
}, 0);
```

```
console.log("Hello");
```

...

Output:

...

Hello

Timer executed

...

\* Poll Phase:

```
```javascript
```

```
const fs = require("fs");
```

```
fs.readFile("file.txt", () => {  
  console.log("File read done");
```

```
});
```

...

The callback goes into the poll phase queue.

Full Flow Example

```
```javascript
```

```
const fs = require("fs");
```

```
console.log("Start");
```

```
setTimeout(() => {
 console.log("Timer");
```

```
}, 0);
```

```
setImmediate(() => {
 console.log("Immediate");
```

```
});
```

```
fs.readFile(__filename, () => {
 console.log("File Read");
```

```
});
```

```
console.log("End");
```

```
...
```

Possible Output:

```
...
```

Start

End

Timer / Immediate (order may vary!)

File Read

```
...
```

Call Stack, Callback Queue, and Microtask Queue

Call Stack (Execution Engine)

- \* A stack (LIFO) where functions are executed.

- \* JavaScript is single-threaded, so only one thing runs at a time.

- \* Example:

```
```javascript
```

```
function a() {
```

```
  console.log("A");
```

```
}
```

```
function b() {
```

```
  a();
```

```
  console.log("B");
```

```
}
```

```
b();
```

```
...
```

Output:

```
...
```

A

B

...

Callback Queue (Macrotask Queue)

* Stores `setTimeout`, `setInterval`, I/O callbacks, and `setImmediate`.

* How it works:

1. Async task finishes (like timer).
2. Its callback goes into the callback queue.
3. Event loop waits for the call stack to be empty.
4. Then pushes it to the stack.

* Example:

```
```javascript
setTimeout(() => {
 console.log("Timer");
}, 0);
```

```
console.log("Hello");
```

...

Output:

...

Hello

Timer

...

## Microtask Queue (HIGH PRIORITY)

\* What goes here: `Promise.then()`, `Promise.catch()`, `queueMicrotask()`, and `process.nextTick()` (Node.js special).

\* Important Rule: Microtasks run BEFORE the callback queue and run IMMEDIATELY after the current stack is empty.

\* Example:

```
```javascript
setTimeout(() => console.log("Timer"), 0);
```

```
Promise.resolve().then(() => {
```

```
console.log("Promise");  
});
```

```
console.log("End");  
...
```

Output:

```
...
```

End

Promise

Timer

```
...
```

Priority Order

1. Call Stack
2. `process.nextTick()`
3. Microtask Queue (Promises)
4. Callback Queue (Timers, I/O)

Golden Rules

- * Rule 1: If the stack is not empty, NOTHING runs.
- * Rule 2: After the stack is empty, run ALL microtasks.
- * Rule 3: Then, take ONE callback from the callback queue.

Real-World Analogy

Think of:

- * Call Stack ? Chef cooking
- * Microtasks ? Urgent VIP orders
- * Callback Queue ? Normal orders

The chef finishes the current dish, immediately handles VIP orders, and then goes to the normal queue.

Summary

- * Call Stack = where code runs
- * Microtask Queue = high priority async (Promises)
- * Callback Queue = lower priority async (Timers, I/O)
- * `process.nextTick()` = highest priority in Node.js